**INSTITUTO TECNOLÓGICO DE COSTA RICA**
**Algoritmos y Estructuras de Datos 1**

# Monopoly

---

Jason Vargas Arce
2024240152

Profesor: Santiago Gamboa Ramirez

II Semestre 2026

# 1. Introducción

Este documento describe el diseño e implementación de una versión simplificada de Monopoly Electrónico (tema clásico, edición Atlantic City en español) desarrollada en C# sobre .NET 8. El sistema sigue una arquitectura cliente-servidor, la computadora del organizador funciona como el banco (servidor) y cada jugador se conecta desde su propia computadora (cliente).

El proyecto tiene tres características centrales: todas las colecciones del juego se implementan con estructuras de datos lineales propias (listas enlazadas, listas circulares y colas), la lógica del juego está separada de la red y de la interfaz gráfica, y se integra un cajero electrónico basado en una Raspberry Pi Pico W con lector RFID, botón para lanzar los dados y LED de confirmación de pago.

El documento presenta las reglas implementadas, la arquitectura de la solución, las estructuras de datos utilizadas, el diagrama de clases, el protocolo de comunicación y las instrucciones de ejecución.

# 2. Descripción del sistema

## 2.1 Objetivo

Permitir que entre 2 y 4 jugadores participen en una partida de Monopoly desde computadoras distintas en la misma red. El servidor mantiene el estado oficial de la partida (saldos, posiciones, propiedades, turnos e historial) y los clientes solo envían solicitudes; nunca modifican el estado directamente.

## 2.2 Reglas principales implementadas

| Concepto | Regla |
|---|---|
| Jugadores | De 2 a 4 por partida. El primer jugador conectado es el organizador y es quien inicia la partida. |
| Saldo inicial | ₡1 500 por jugador. |
| Tablero | 40 casillas recorridas de forma circular: propiedades, ferrocarriles, compañías de servicio, Casualidad, Arca Comunal y casillas especiales. |
| Premio de salida | ₡200 cada vez que el jugador pasa por la casilla de Salida. |
| Compra | Al caer en una propiedad sin dueño, el jugador decide si la compra; la compra se confirma con su tarjeta. |
| Alquiler | Al caer en una propiedad ajena se paga alquiler al propietario (calculado por cada tipo de propiedad). |
| Cárcel | La casilla: Vaya a la cárcel hace perder 1 turno. |
| Cartas de evento | Casualidad y Arca Comunal: la carta usada vuelve al final del mazo. |
| Eliminación | Si un jugador no puede cubrir un pago obligatorio, paga lo que tiene y queda eliminado. |
| Fin de partida | Cuando queda un solo jugador activo o al completarse el máximo de turnos (100 por defecto); en ese caso gana quien tenga mayor patrimonio. |

### 2.3 Modos de juego

- **Modo hardware (por defecto):** los dados se lanzan con el botón físico del cajero y las compras y pagos se confirman acercando la tarjeta RFID del jugador.
- **Modo sin hardware (pruebas):** el organizador simula el botón y la tarjeta desde su ventana, lo que permite jugar y probar el sistema sin la placa.

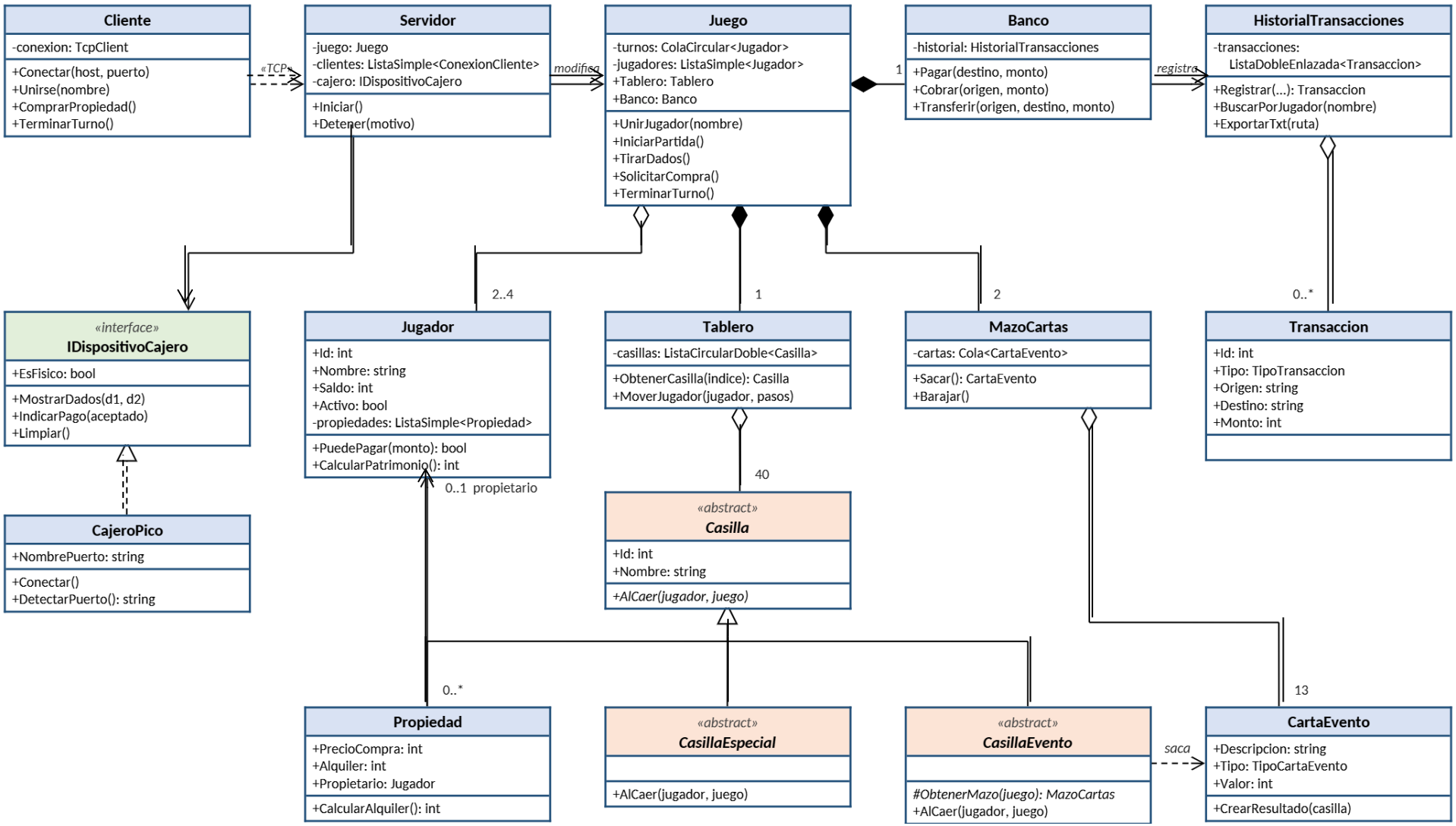
## 3. Estructuras de datos

El enunciado exige no utilizar las colecciones de .NET, por lo que se implementaron las siguientes estructuras genéricas desde cero:

| Estructura | Implementación | Uso en el juego | Justificación |
|---|---|---|---|
| ListaSimple<T> | Nodos simples con referencia a cabeza y cola | Propiedades de cada jugador; clientes del servidor | Solo se agrega, busca y recorre hacia adelante. |
| ListaDobleEnlazada<T> | Nodos dobles con cabeza y cola | Historial de transacciones | Permite recorrer de la más antigua a la más reciente y al revés. |
| ListaCircularDoble<T> | Nodos dobles enlazados en círculo | Tablero de 40 casillas | Después de la última casilla sigue la primera; el jugador avanza nodo a nodo. |
| ColaCircular<T> | Arreglo con índices modulares | Orden de turnos | Rotar al siguiente jugador es O(1) y permite eliminar jugadores. |
| Cola<T> | Nodos simples, FIFO | Mazos de Casualidad y Arca Comunal | Se saca la carta del frente y vuelve al final. |

En el código, cada estructura está acompañada de su clase de nodo (NodoSimple<T>, NodoDoble<T>, NodoCircularDoble<T>). El detalle de operaciones y complejidades está en docs/estructuras.md del repositorio.

# 4. Diagrama de clases



## 4.1 Cómo leer el diagrama

El diagrama muestra las clases principales con sus atributos y métodos más importantes (no todos). Los símbolos de visibilidad son: público, privado y protegido. Las clases en cursiva o con «abstract» no se pueden instanciar directamente. Todas las figuras se pueden mover y editar en Word: cada clase es un grupo, y su texto se modifica haciendo clic dentro de la caja.

| Relación | Notación | Ejemplo en el proyecto |
|---|---|---|
| Herencia | Línea continua con triángulo vacío | Propiedad, CasillaEvento y CasillaEspecial heredan de Casilla. |
| Realización | Línea punteada con triángulo vacío | CajeroPico implementa la interfaz IDispositivoCajero. |
| Composición | Rombo relleno en el lado del «todo» | Juego contiene su Tablero, su Banco y dos MazoCartas; no existen sin el juego. |
| Agregación | Rombo vacío en el lado del «todo» | Juego agrupa de 2 a 4 Jugador; el Tablero agrupa 40 Casilla. |
| Asociación | Línea continua con flecha abierta | Servidor usa Juego; Banco registra en el HistorialTransacciones. |
| Dependencia | Línea punteada con flecha abierta | Cliente se comunica con Servidor por TCP; CasillaEvento usa CartaEvento. |

## 4.2 Descripción de las clases

| Clase | Responsabilidad |
|---|---|
| Juego | Estado oficial de la partida: jugadores, turnos, fases, validaciones y efectos de casillas y cartas. Es seguro para varios hilos. |
| Banco | Única clase que mueve dinero (pagar, cobrar y transferir); cada operación queda registrada como transacción. |
| Tablero | Las 40 casillas en una lista circular doble; mueve a los jugadores y avisa cuando pasan por Salida. |
| Jugador | Datos del jugador: nombre, saldo, posición, propiedades, tarjeta RFID y si sigue activo. |
| Casilla | Clase abstracta base. Cada subclase define qué ocurre al caer en ella mediante AlCaer (polimorfismo). |
| Propiedad | Casilla que se puede comprar; calcula su alquiler. Ferrocarril y CompaniaServicio heredan de ella. |
| CasillaEvento | Casillas que sacan una carta: CasillaCasualidad y CasillaArcaComunal. |
| CasillaEspecial | Salida, Impuesto, Cárcel (solo visita), Parada libre y Vaya a la cárcel. |
| MazoCartas / CartaEvento | Mazo implementado con Cola<T> y las cartas con su tipo y valor. |
| HistorialTransacciones / Transaccion | Registro de todos los movimientos de dinero; permite filtrar y exportar a TXT. |
| Servidor / Cliente | Comunicación TCP. El servidor es el único que modifica Juego; el cliente solo envía solicitudes y recibe eventos. |

| Clase | Responsabilidad |
|---|---|
| IDispositivoCajero | Interfaz del cajero electrónico; la implementan CajeroPico (placa real) y CajeroSimulado (pruebas). |

# 5. Protocolo de comunicación

Cliente y servidor intercambian un mensaje por línea en texto con comando.

| Solicitud (cliente → servidor) | Descripción |
|---|---|
| CONECTAR | Une al jugador a la partida (opcionalmente con forma y color de ficha). |
| INICIAR_PARTIDA | El organizador inicia la partida (mínimo 2 jugadores). |
| TIRAR_DADOS | Lanza los dados (solo en modo sin hardware; si no, se usa el botón físico). |
| COMPRAR_PROPIEDAD / NO_COMPRAR | Decide sobre la propiedad en la que cayó el jugador. |
| TERMINAR_TURNO | Pasa el turno al siguiente jugador. |
| CONSULTAR_TRANSACCIONES\|filtro | Pide el historial: todas, por jugador, por tipo, antiguas o recientes. |
| EXPORTAR_TRANSACCIONES | Genera el archivo TXT de la partida en la computadora del servidor. |

El servidor responde con BIENVENIDA, ERROR, EVENTO, ESTADO, DADOS, TRANSACCIONES y FIN

# 6. Módulo de hardware

El cajero electrónico es una Raspberry Pi Pico W programada en MicroPython y conectada por USB a la computadora del organizador. Incluye un lector RFID RC522 para identificar a los jugadores con tarjetas, un botón para lanzar los dados y un LED que indica si un pago fue aceptado. La placa y la computadora se comunican por puerto serie con mensajes de texto simples.

Gracias a la interfaz, el servidor no depende de la placa concreta: puede trabajar con CajeroPico o con CajeroSimulado sin cambiar la lógica del juego.

# 7. Conclusiones

- Implementar las estructuras lineales desde cero permitió elegir la más adecuada para cada necesidad: la lista circular doble modela naturalmente el tablero y la cola circular hace eficiente la rotación de turnos.
- La herencia y el polimorfismo en la jerarquía de Casilla evitan condicionales por tipo de casilla y facilitan agregar casillas nuevas.
- Mantener el estado oficial únicamente en el servidor simplifica la sincronización entre jugadores y evita inconsistencias.
- La interfaz IDispositivoCajero desacopla el hardware de la lógica y permite probar el juego completo sin la placa.